Computers & Security 154 (2025) 104451

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

Bayes-based word weighting for enhanced vulnerability classification in critical infrastructure systems

Aissa Ben Yahya *, Hicham El Akhal, El Mehdi Ismaili Alaoui, Abdelbaki El Belrhiti El Alaoui

Computer Science and Applicatioins Laboratory, Faculty of Sciences, Moulay Ismail University of Meknes, PB 11201 Zitoune, 50000 Meknes, Morocco

ARTICLE INFO

Dataset link: https://nvd.nist.gov/vuln/data-feeds, https://www.cnnvd.org.cn/home/dataDownLoad, https://www.cnvd.org.cn/shareData/download/243, https://www.variotdbs.pl/vuln s/, https://github.com/aissa302/Bayesian-weighting-framework.git

Keywords:
Word weighting
Vulnerability classification
Critical infrastructure security
Deep learning
Word embedding
Text classification

ABSTRACT

The increasing number of vulnerabilities in embedded devices poses a significant threat to the critical infrastructure security where these devices are used. While deep learning approaches have advanced software vulnerability classification, they exhibit critical limitations regarding word weighting. Conventional methods like term frequency–inverse document frequency (TF–IDF) prioritize global term distributions but overlook intra-class distinctions. While improved variants of this technique have been proposed, they often fail to consider that a word's importance can vary across categories and struggle to prioritize rare but distinctive words adequately. Additionally, high inter-class semantic overlap and terminological ambiguity in vulnerability descriptions hinder model performance by failing to separate intra-class keywords From background noise. to address these gaps, we propose a novel vulnerability classification and word vector weighting approach based on bayes theorem. our method dynamically adjusts term relevance by calculating posterior probabilities of word-category associations, emphasizing rare tokens with high intra-class specificity. we validate the approach on four test datasets derived from databases such as the national vulnerability database (NVD) and the chinese vulnerability database (CNNVD). rigorous ablation and comparative studies demonstrate that bayes-based word weighting outperformed other methods by achieving a performance of 97.63% accuracy, and 97.60% F1-score on the most challenging test data. all our models and code to produce our results are open-sourced.

1. Introduction

Critical infrastructure security increasingly depends on embedded devices found in the Internet of Things (IoT), Industrial Control Systems (ICS), healthcare, and automotive sectors to name a few. These devices often lack comprehensive security measures or have not undergone sufficient vulnerability testing. This concern is reflected in the Cybersecurity and Infrastructure Security Agency (CISA) ICS Advisories,[1] which have reported 2459 alerts related to ICS devices and software, with 1243 alerts rated with a Common Vulnerability Scoring System (CVSS) severity of Medium or higher.[2] Additionally, initiatives such as CISA's Secure-by-Design and Secure-by-Default principles[3] underscore the critical need to bolster the security posture of these devices. This urgency is further underscored by initiatives like MITRE's EMB3D framework (EMB3D, 2024), a threat modeling effort focused exclusively on embedded devices, which emphasizes the need for precise vulnerability classification to guide mitigation strategies (Jian et al., 2017).

Several studies have been done in the literature regarding the automation of software vulnerability classification taking advantage of the recent advancements in machine learning (ML) and semiconductor technology. These studies have explored different areas such as predicting the vulnerability severity (Han et al., 2017; Kudjo et al., 2019), mapping Common vulnerability and exposure (CVE) to Common weaknesses enumeration (CWE) (Wang and Guo, 2010; Vishnu et al., 2022), extracting embedded systems vulnerabilities (Ben Yahya et al., 2024), classifying computer vulnerabilities and network security (Liu, 2025), categorizing IoT appliances (Blinowski and Piotrowski, 2020), vulnerability reproducibility (Davari et al., 2017). These studies have seen remarkable success and effectively reduced the time gap between the report of vulnerabilities and their analysis, contributing to the

* Corresponding author.
E-mail addresses: ai.benyahya@edu.umi.ac.ma (A. Ben Yahya), hi.elakhal@edu.umi.ac.ma (H. El Akhal), e.ismailialaoui@umi.ac.ma (E. Mehdi Ismaili Alaoui), a.elbelrhitielalaoui@umi.ac.ma (A.E.B. El Alaoui).

[1] https://www.cisa.gov/news-events/cybersecurity-advisories.
[2] https://www.icsadvisoryproject.com.
[3] https://www.cisa.gov/sites/default/files/2023-06/principles_approaches_for_security-by-design-default_508c.pdf.

https://doi.org/10.1016/j.cose.2025.104451
Received 14 December 2024; Received in revised form 19 February 2025; Accepted 17 March 2025
Available online 25 March 2025
0167-4048/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

advance in the field of cyber threat analysis. However, the accurate classification heavily depends on the effective text representation, which mainly consists of word embedding, a phase in which the words are represented as vector space, and word weighting, which represents the importance of each word for the classification task.

In early studies, the document-term matrix (TDM) (Wen et al., 2015; Spanos and Angelis, 2018) was used for text representation, sometimes coupled with feature extraction techniques (e.g., chi-square) for dimensionality reduction. However, this approach fails to account for the importance of terms across the entire corpus because it merely calculates the frequency of words in a document. Consequently, several works have adopted term frequency–inverse document frequency (TF–IDF) (Salton and Buckley, 1988) as an alternative, since it more effectively gauges the importance of terms across the corpus. For instance, Huang et al. (2019) utilized TF–IDF, Information Gain (IG), and Deep Neural Networks (DNN) to classify vulnerabilities by type, achieving an accuracy of 87%. Similarly, Aghaei et al. (2020) employed TF–IDF to assign weights to the extracted n-gram features when mapping CVEs to CWEs.

While TF–IDF demonstrated superiority over the document-term matrix in terms of text representation and word weighting, it prioritizes global word distribution while neglecting intra-class discriminative power. In addition, it ignores the co-occurrence of words, their similarities, and contextual information. These limitations led to the emergence of word embedding techniques such as Word2Vec (Mikolov et al., 2013), GloVe (Pennington et al., 2014), and FastText (Bojanowski et al., 2017). For example, Han et al. (2017) proposed a method for predicting vulnerability severity using Word2Vec and CNN. Word2Vec outperformed traditional techniques like TF–IDF due to its ability to capture contextual relationships and term similarities, thereby improving classification performance. Further, Sharma et al. (2021) introduced a vulnerability prioritization system that determines the severity level of a vulnerability based on its description. They employed pre-trained GloVe for word embedding and CNN for classification. Although pre-trained word embedding techniques achieved better performance than statistical word embeddings, since they capture richer information, the absence of a weighting mechanism reduced their effectiveness in keyword-driven classification tasks, such as vulnerability classification. Additionally, these techniques yield generic word embeddings that lack domain-specific information, and the out-of-vocabulary issue remains a significant challenge.

To address these limitations, researchers have explored hybrid methods that combine word embeddings with word weighting to enhance classification accuracy. For example, Vishnu et al. (2022) proposed a system using a self-attention neural network with DNN and GloVe embeddings for type-based vulnerability classification. The self-attention layer enabled the model to focus on the most relevant parts of the document, thereby improving accuracy. Similarly, Wang et al. (2023) proposed N-TF–IDF an advanced version of TF–IDF for word weighting, utilized a domain-specific Word2Vec for word embedding, and introduced a hybrid neural network combining TextCNN and BiGRU to classify vulnerabilities based on their CWE-defined types. Despite the significant success of these approaches, they often fail to consider that a word's importance can vary across categories and struggle to adequately prioritize rare but distinctive words.

Recently, transformers (Vaswani, 2017) have become the leading approach for text representation, achieving state-of-the-art results in various NLP tasks, including text classification (Gillioz et al., 2020). Transformers have shifted word embedding from an independent state, where words do not interact (e.g., Word2Vec, GloVe, FastText), to a fully contextualized state, where words exchange information and influence each other. Researchers in software vulnerability classification have leveraged this advancement, applying transformers (Shahid and Debar, 2021; Zhu et al., 2022; Marali et al., 2024; Ezenwoye and Pinconschi, 2024; Sun et al., 2024) to classify vulnerabilities. Despite their impressive performance, transformer-based models heavily rely on contextual information and sometimes prioritize background words over category-important words. This limitation makes them less effective for keyword-driven classification tasks.

While existing classification methods are widely used and effective, they often do not specifically address the security of critical infrastructure embedded devices. As a result, there remains an inconsistent understanding of the security threats these systems face and the appropriate mechanisms or capabilities to mitigate them. To bridge this gap and overcome the limitations of current word-weighting methods, this paper proposes a domain-based classification approach coupled with a novel word-weighting method grounded in Bayes' theorem. The domain-based classification framework categorizes vulnerabilities into those related to critical infrastructure embedded devices and those related to general-purpose systems. The proposed automatic vulnerability classification algorithm employs a Word2Vec model trained on custom cybersecurity-specific data to produce low-dimensional, dense word embedding vectors. Bayes' theorem is then used to construct more effective weighted word vectors, enhancing the model's ability to emphasize terms that are particularly relevant to the classification of vulnerabilities. The current study provides the following key contributions:

- **Domain-Based Classification:** A novel classification framework that distinguishes vulnerabilities related to critical infrastructure embedded devices from those of general-purpose systems.
- **Novel Word Weighting Method:** An innovative Bayes' theorem-based approach that dynamically adjusts term relevance by assigning distinct weights to the same word across different categories, thereby emphasizing rare words with high intra-class specificity.
- **Domain-Specific Word Embedding:** The use of a Word2Vec model trained on custom cybersecurity data to create contextually rich, domain-specific word embeddings for improved text representation.

Our paper is organized into several sections: Section 2 reviews related works, Section 3 details the proposed approach in this work, and in Section 4, we present and discuss the evaluation results of this mechanism. The limiations and future work are presented in Section 5. Finally, we draw our conclusions in Section 6, summarizing the insights gathered from the paper's content.

2. Related work

The classification of software vulnerabilities and the detection of ambiguity in natural language descriptions have seen significant advancements through ML and DL techniques. Existing approaches broadly fall into three categories: (1) traditional feature engineering with shallow classifiers, (2) deep neural networks with static embeddings, and (3) Hybrid and attention-based approaches. Below, we critically analyze these methods, their limitations, and how they motivate our proposed approach.

2.1. Traditional feature engineering methods

Early studies focused on term-frequency-based weighting and dimensionality reduction. For instance, Wen et al. (2015) employed a document-term matrix (DTM) with chi-square feature selection to address high dimensionality. Similarly, Spanos and Angelis (2018) proposed a multi-label classification model to assign vulnerability characteristics (e.g., attack vector, impact) using DTM for word embedding and weighting. While effective for simple term distributions, this approach fails to capture semantic relationships or contextual word importance, leading to suboptimal performance on domain-specific texts. Similarly, Huang et al. (2019) combined TF–IDF with information gain (IG) and deep neural networks (DNNs). However, TF–IDF assigns

global weights, ignoring category-specific term relevance, while IG prioritizes globally discriminative features rather than domain-localized ones. These limitations reduce their effectiveness in distinguishing fine-grained vulnerability categories. In the same context, Aghaei et al. (2020) extended this paradigm with ThreatZoom, a framework mapping CVEs to CWEs using n-gram features weighted by TF–IDF. Although n-grams capture local word sequences, the approach inherits the same issues of TF–IDF weighting technique. Ben Yahya et al. (2024) further exemplify the limitations of global term weighting, employing SVM with TF–IDF to classify embedded system vulnerabilities. While TF–IDF effectively suppresses common terms (e.g., "vulnerability" or "system"), its inability to prioritize category-specific keywords (e.g., "TP-Link" in critical infrastructure devices, and "Windows" in general-purpose systems) reduces precision in multi-domain settings. This reinforces the need for adaptive weighting schemes that tailor term importance to individual vulnerability categories.

2.2. Deep learning with static embeddings

Recent work has leveraged neural networks with word embeddings. Han et al. (2017) trained a vulnerability-specific skip-gram model and used CNNs to capture sentence-level context. While their embeddings improved semantic relevance, the lack of term weighting mechanisms caused background words to dominate classification. Sharma et al. (2021) used pre-trained GloVe embeddings with CNNs but suffered from domain mismatch, as pre-trained word embeddings poorly represent specialized vulnerability terms. Character-level approaches, such as Nakagawa et al. (2019), avoided out-of-vocabulary issues but discarded semantic relationships by using one-hot encoding. These methods highlight a critical gap: the need for domain-adaptive embeddings that balance semantic relevance and term importance.

2.3. Hybrid and attention-based approaches

Hybrid models aim to integrate feature selection with contextual awareness. Wang et al. (2022) combined word2vec with CNN-GRU architectures to capture local and global textual patterns. Despite their success, the absence of explicit term weighting allowed non-discriminative terms to influence predictions. Vishnu et al. (2022) introduced a self-attention DNN (SA-DNN) to prioritize critical words. However, self-attention mechanisms are sensitive to noise and often might prioritize background words, reducing robustness. Notably, Kudjo et al. (2019) and Chen et al. (2020) proposed TF-IGM weighting, which emphasizes rare yet significant terms. While TF-IGM improves over TF–IDF, its fixed per-term weights across classes risk misclassifying terms shared between domains. Wang et al. (2023) advanced TF–IDF with their N-TF–IDF variant, which weights terms by their importance within individual vulnerability classes rather than globally. Combined with a TextCNN-BiGRU hybrid network, their approach improved domain-specific classification. However, like TF-IGM, N-TF–IDF struggles to prioritize rare but critical terms, as it retains fixed term weights across categories. This underscores the need for dynamic weighting that adapts to both term rarity and category-specific relevance. Recent works have also explored transformer-based architectures for vulnerability classification. Kota et al. (2024) leveraged BERT cross-encoders to model hierarchical relationships between CVE descriptions and CWE definitions, capturing deep semantic context. While BERT excels at contextual reasoning, its reliance on holistic sentence embeddings risks overlooking domain-specific keywords (e.g., "Firmware", "D-Link" or "BIOS") critical for keyword-driven classification. Our method bridges this gap by augmenting semantic embeddings with Bayesian term weighting, ensuring both contextual and keyword-centric signals are prioritized.

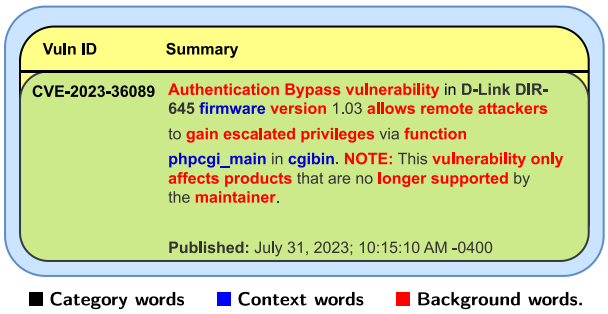


Fig. 1. An example of a vulnerability description reported in the NVD database and the type of words it consists of. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

2.4. Research gaps and motivation

Current methods face three key challenges: (1) Domain-specific semantic gaps: Pre-trained or generic embeddings inadequately represent vulnerability terminology. (2) Static term weighting: TF–IDF and TF-IGM lack adaptability to class-specific term importance. (3) Contextual noise: Models like CNNs, SA-DNNs, or transformer-based, struggle to suppress irrelevant terms in vulnerability descriptions. Our approach addresses these gaps through a Bayes' theorem-based weighting mechanism that dynamically adjusts term importance per vulnerability domain, coupled with domain-tuned embeddings to capture semantic and syntactic nuances.

3. Proposed approach

3.1. Word classification in vulnerability descriptions

Text classification in natural language processing (NLP) involves categorizing documents based on their content by identifying specific keywords or patterns that indicate their relevance to a particular category. Experts typically search for *category-defining words* that provide clear evidence of a document's classification. When such definitive words are absent, experts rely on *context words*, which are associated with a specific category but may appear in others.

In our study, we extend this approach by categorizing words in vulnerability descriptions into three types: *category words, context words,* and *background words*.

- ***Category words*** provide near-certain classification (around 100%) that a document d belongs to category C.
- ***Context words*** suggest association with category C with a certainty above 50% but below 100%.
- ***Background words*** offer minimal classification insight as they do not significantly impact a classifier's certainty about the category.

Fig. 1 illustrates an example from an NVD-reported vulnerability, specifically in the context of domain-type classification, distinguishing between embedded device-related vulnerabilities and general-purpose system vulnerabilities, highlighting different word types: *category words* (e.g., *D-Link, DIR-645*) in bold black, which are definitive for classification as an embedded device vulnerability; *context words* (e.g., *firmware*) in bold blue, indicating likely association; and *background words* (e.g., *Authentication, bypass, allow, attackers*) in bold red, providing little to no category-specific information.

By weighting words in a way that differentiates between category, context, and background words, our method enhances the identification of ambiguous texts and mitigates the effects of category overlap,

particularly in the context of domain-type classification. To represent our data in this structured format, we developed a novel word weighting method based on Bayes' theorem, which calculates word weights according to their contribution to a model's certainty. Words unique to a category receive the highest weights, while those frequently occurring within one category but rarely found in others are assigned intermediate weights. Background words receive the lowest weights, reflecting their minimal impact on classification certainty.

3.2. Word weighting using Bayes theorem

The traditional TF–IDF algorithm does not consider the distribution of words among different categories, treating all texts in the dataset as a single category, overlooking intra-class distinctions, and assigning only one weight to each word (Guo et al., 2019). However, a word's importance in classification tasks may vary across different categories. Chen et al. (2016) proposed Term Frequency & Inverse Gravity Moment (TF-IGM), which fully utilizes the fine-grained term distribution across different text classes. While IGM prioritizes words with high discriminative power despite their rarity, it does not account for the inter-class weight variance of the same word. In the same context (Wang et al., 2023) proposed N-TF–IDF to account for the distribution of words within each category, assigning different weights to the same word across different categories. N-TF–IDF demonstrated its superiority and effectiveness in vulnerability text classification. However, it assigns low weights to words that are unique to a category but appear in only a few texts within that category. In addition, their formula (1)

$$Weight_Vector(t_i) = Word2Vec(t_i) * max(NTFIDF_{i,j}) \tag{1}$$

multiplies the word vector generated by Word2Vec with the maximum N-TF–IDF weight across all categories each time a word is encountered, resulting in a fixed weight for the word across categories rather than preserving category-specific distinctions. Our proposed word weighting technique addresses this issue by multiplying the word vector by the weight of the corresponding category, ensuring that unique words in a category receive high weights regardless of their frequency within that category.

To use Bayes' theorem for word weighting, we first calculate the prior probability of each category, the likelihood, and the prior of each word. The prior of each category can be calculated as shown in formula (2)

$$p(c_j) = \frac{\sum_k d_{k,j}}{D} \tag{2}$$

where $p(c_j)$ is the prior and it is the probability of a document in the corpus being category c_j, and $\sum_k d_{k,j}$ is the total number of documents in the category c_j, while D is the total number of documents in the corpus.

The likelihood can be calculated as shown in formula (3)

$$p(w_i|c_j) = \frac{\sum_i d_{i,j}}{D_j} \tag{3}$$

where $p(w_i|c_j)$ is the likelihood and it is the probability of a document in the corpus containing the word w_i given that its category is c_j, $\sum_i d_{i,j}$ is the total number of documents containing word w_i in category c_j, D_j is the total number of documents in category c_j.

The prior of each word can be calculated as depicted in formula (4)

$$p(w_i) = \frac{\sum_m d_m}{D} \tag{4}$$

where $p(w_i)$ is the probability of a document in the corpus containing the word w_i, $\sum_m d_m$ is the total number of documents containing the word w_i, D is the total number of documents in the entire corpus.

To calculate the weight of w_i in category c_j we used the Bayes theorem as shown in formula (5):

$$p(c_j|w_i) = \frac{p(w_i|c_j) * p(c_j)}{p(w_i)} \tag{5}$$

where $p(c_j|w_i)$ is the posterior and it is the probability of a document being category c_j given that it contains the word w_i and it is also the weight of the word w_i in category c_j

3.3. Word vector weighting

Although the Word2Vec model considers the local relationships of contextual words, it overlooks the importance of specific words in different text categories, as it assigns the same vector to a word regardless of the category. Weighting the word vectors generated by the Word2Vec model can better capture the category-specific significance of words (Tian and Wu, 2018).

In this paper, we use Bayes-based word weights to enhance domain-specific Word2Vec embeddings. Specifically, we first construct a weight matrix $W \in \mathbb{R}^{|U|\times|C|}$ using a Bayes theorem-based algorithm, where $|U|$ is the number of unique words in the corpus and $|C|$ is the number of categories. This matrix represents the weight of each word across different categories. For a given word, its weight in the corresponding category is selected and multiplied by its Word2Vec vector to obtain the final weighted word vector.

However, because the probabilities obtained from Bayes' theorem are constrained between 0 and 1, directly using these probabilities as weights has minimal impact when multiplying them with Word2Vec vectors—the maximum scaling factor being 1. To address this, we apply a log-odds transformation to the weights before multiplying them with the word vectors. This transformation amplifies differences in probabilities, ensuring that the weighted word vectors more effectively reflect the category-specific importance of words. The formula for obtaining the final weighted word vectors is shown in Formula (7).

$$W_{i,j} = \log\left(\frac{p(c_j \mid w_i)}{1 - p(c_j \mid w_i)}\right) \tag{6}$$

$$\mathrm{V}_{i,j} = \mathrm{W2V}(w_i) \cdot W_{i,j} \tag{7}$$

where $W_{i,j}$ is the weight of word w_i in category c_j, $V_{i,j}$ is the weighted word vector of word w_i in category c_j, $\mathrm{W2V}(w_i)$ is the word embedding vector of word w_i.

3.4. Domain-based vulnerability classification

The automatic domain-based vulnerability classification proposed in this paper is based on converting words into vectors using domain-specific word2vec and weighting these vectors using Bayes-based weighting. The overall steps of the framework are shown in Fig. 2

3.4.1. Data labeling

After downloading the data, we employed a Python script[4] to label it. The script contains two keyword lists: one for critical infrastructure embedded devices, and another for general-purpose systems.

The process of acquiring the keyword lists involved an extensive search of online resources and the use of generative LLMs to identify terms associated with embedded devices, protocols, and software employed in critical infrastructures. A similar approach was used to compile terms for general-purpose systems. A subset of these keyword lists is presented in Table 1. Based on the collected keywords, our script scans each vulnerability description for occurrences from both lists.

[4] https://github.com/aissa302/CVE-Aanlysis-Project.

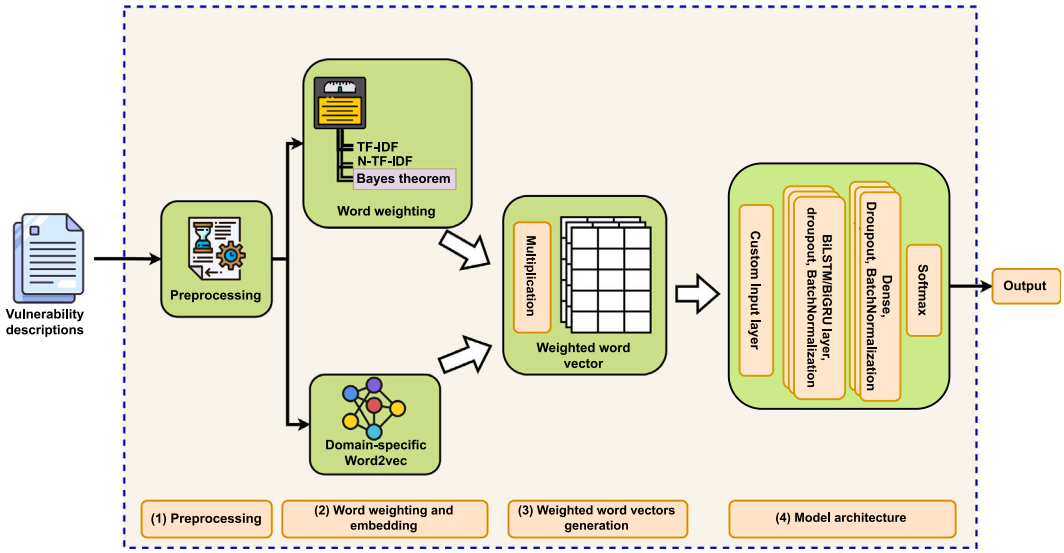


Fig. 2. System architecture of the proposed vulnerability classifier.

Table 1
ESV and GPSV list of keywords.

ESV	GPSV
Juniper ACX	Windows desktop
NETGEAR Nighthawk	Dell XPS 13 g.
ASUS RT-AX88U	Brave Browser
DIR-645 Wired	Windows Bluetooth Driver
Grandstream GXV3611_HD camera	Apple OS X
Amazon Echo	baserCMS
August Doorbell	Image Magick
NX-OS	Huawei FusionServer
D-Link	IBM System x
Microcontroller	Dell PowerEdge

If a description contains any critical infrastructure (CI) keywords and no general-purpose system (GPS) keywords, it is labeled as an "ESV" (Embedded System Vulnerability). Conversely, if it contains any GPS keywords and no CI keywords, it is labeled as a "GPSV". This approach is inspired by the work in Papp et al. (2015), which employed a similar methodology for vulnerability classification.

3.4.2. Data preprocessing

To prepare the data for modeling, we applied the following preprocessing steps:

- *Translation:* We translated data collected from Chines databases, from Chinese to English using Google Translate API (Google translate API, 2024).
- *Lowercase conversion:* We converted all text to lowercase to reduce the number of unique features and improve model performance.
- *Digit removal:* We removed all digits from the text because they do not carry any significant meaning when it comes to classification, also that lets the model focus on the linguistic features of the data.
- *Contraction expansion:* We expanded contractions (e.g., "you're" to "you are", "I'm" to "I am") to improve tokenization.
- *Stopword removal:* We removed common stopwords, such as "the", "and", etc., that do not add significant value to the text, to improve model accuracy.
- *Porter Stemming:* We applied Porter Stemming to reduce words to their base form.
- *Special character removal:* We removed special characters from the text to focus on the linguistic features of the data.
- *Base form conversion:* We converted words to their base form (e.g., "ran" to "run") to further reduce dimensionality.

3.4.3. Ambiguous data augmentation

In our domain-based vulnerability classification, we encountered two primary types of ambiguity. The first arises when vulnerability descriptions lack specific keywords, making classification challenging. The second occurs when a vulnerability relates to a system that is not unique to any category such as programming languages, common libraries, or protocols. An example of vulnerability descriptions and their corresponding ambiguity type is shown in Table 2. Since these types of vulnerabilities cannot be identified using category keywords alone, we manually collected samples for these categories. However, because manual collection is time-consuming, only a few samples were obtained, resulting in a significant imbalance between the other categories and the ambiguous category. To address this issue, we employed NLP augmentation techniques to balance our dataset Section 3.5.

3.4.4. Text representation

Once the vulnerability texts are preprocessed, the Bayes-based weighting and the domain-specific Word2Vec model are employed to construct weighted word vectors. To leverage the benefits of the Bayes-based weighting scheme, we designed a model architecture comprised of interconnected components designed for efficient vulnerability classification. It begins with an Input Layer that processes weighted word vectors generated using Word2Vec and Bayes-based weighting. A BiLSTM/BiGRU Layer follows, capturing forward and backward dependencies to encode contextual information. Dropout and Batch Normalization are applied to enhance robustness and training stability. A Dense Layer reduces dimensionality while maintaining generalization through additional dropout and normalization. Finally, a Softmax Layer calculates class probabilities, ensuring accurate and interpretable classification across predefined categories.

The algorithm below Algorithm 1 outlines the steps for preparing the input data in the required format. The key steps involve splitting the input text into words, looking up the word vectors in the domain-specific Word2Vec model, and applying the Bayes-based weights to the word vectors based on the corresponding category. The resulting weighted word vectors are then padded or truncated to a fixed length to ensure consistent input to the model.

Table 2
Vulnerability descriptions and their corresponding Ambiguity type.

CVE-ID	Description	Ambiguity type
CVE-2023–0755	The affected products are vulnerable to improper validation of array index, which could allow an attacker to crash the server and remotely execute arbitrary code.	Missing keywords
CVE-2023–23415	Internet Control Message Protocol (ICMP) Remote code execution vulnerability.	Common

Algorithm 1 Weighted word vectors preparation

Input: Set of documents $\mathcal{D} = \{d_1, d_2, \ldots, d_m\}$ and their corresponding categories $\mathcal{C} = \{c_1, c_2, \ldots, c_m\}$.
Input: Domain-specific Word2Vec model $w2v_model$.
Input: Bayes theorem-based weights $weights(w, c)$ for word w in category c.
Input: Maximum sequence length max_len.
Input: Embedding dimension d.
Output: Embedding matrix $E \in \mathbb{R}^{m \times max_len \times d}$.
Initialize $E = []$.
for each document $d_i \in \mathcal{D}$ with category $c_i \in \mathcal{C}$ **do**
  Split d_i into words: $d_i = \{w_1, w_2, \ldots, w_{|d_i|}\}$.
  Initialize $\mathbf{e}_{d_i} = []$.
  for each word $w_j \in d_i$ **do**
    if $w_j \in w2v_model.wv$ **then**
      if $weights(w_j, c_i)$ exists **then**
        $\mathbf{v}_{w_j} = w2v_model.wv[w_j] * weights(w_j, c_i)$.
      else
        $\mathbf{v}_{w_j} = w2v_model.wv[w_j]$.
      end if
    else
      $\mathbf{v}_{w_j} = \mathbf{0} \in \mathbb{R}^d$.
    end if
    Append $\mathbf{v}_{w_j}$ to $\mathbf{e}_{d_i}$.
  end for
  if $|\mathbf{e}_{d_i}| < max_len$ **then**
    Pad $\mathbf{e}_{d_i}$ with $\mathbf{0} \in \mathbb{R}^d$ to length max_len.
  else
    Truncate $\mathbf{e}_{d_i}$ to the first max_len vectors.
  end if
  Append $\mathbf{e}_{d_i}$ to E.
end for
Return: E.

3.5. Data augmentation and bias mitigation

One of the primary challenges in embedded devices vulnerability classification is the inherent imbalance in the dataset. Upon analyzing the distribution of vulnerability descriptions in our collected dataset, we identified two significant issues: inner imbalance and outer imbalance.

Inner imbalance refers to the disparity within the data of the same class, where a small subset of keywords (systems, products, or software in our case) dominates the rest. For example, the number of vulnerability descriptions related to a particular system may significantly outnumber those related to other systems within the same class. This imbalance can lead to models that efficiently classify vulnerabilities for systems with more vulnerabilities, such as Android while struggling with minority categories that contain only 10 or fewer vulnerabilities. Consequently, biased model training may occur, causing the model to become overly specialized in recognizing patterns related to the majority systems and neglecting the minority systems, which can result in overfitting and poor generalization performance.

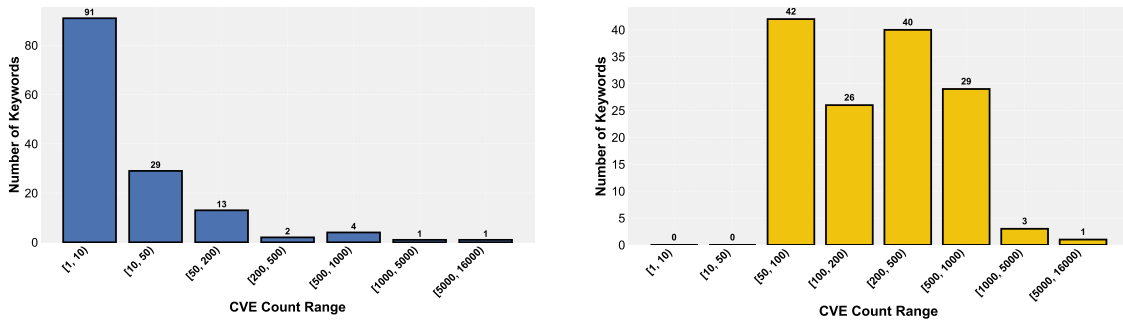
Outer imbalance, on the other hand, refers to the inconsistency in the number of samples between classes. This imbalance can also lead to biased model training, as the model may become overly influenced by the majority class and fail to generalize well to the minority classes.

3.5.1. Mitigating imbalance

To address both inner and outer imbalance issues, we employed a combination of NLP augmentation techniques and a two-stage undersampling strategy. First, data augmentation was performed using two methods. The *cross-lingual translation* approach leveraged identical CVEs from Chinese databases, translating their descriptions into English to generate linguistically diverse representations of vulnerabilities. In addition, we introduced *style variation* by incorporating writing styles from multiple vulnerability databases, thereby reducing bias toward specific linguistic patterns.

The effectiveness of these augmentation techniques is evident in the distribution shift illustrated in Fig. 3 (for critical infrastructure systems) and Fig. 4 (for general-purpose systems). Before augmentation (see Figs. 3(a) and 4(a)), the distributions were severely skewed, with 64% of keywords containing fewer than 10 vulnerability descriptions for critical infrastructure device and 49% contains between 1 and 50 for general-purpose systems. Following augmentation (see Figs. 3(b) and 4(b)), the distribution in both systems leans toward the center between 50 and 1000

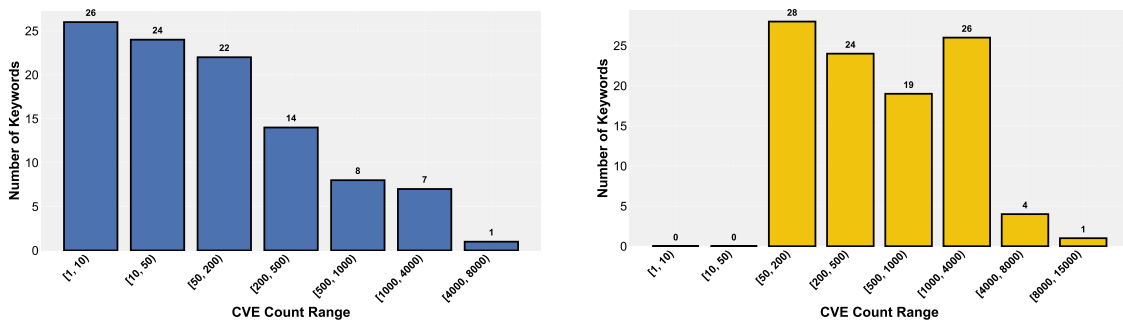
To further mitigate imbalance, a two-stage undersampling strategy was implemented. In the first stage, we applied a *minority protection* approach by retaining all vulnerability descriptions from keywords with fewer than 200 samples to preserve critical minority-class information. In the second stage, for keywords with 200 or more descriptions, we randomly selected 200 samples to limit overrepresentation.



(a) CVE distribution before data augmenataion

(b) CVE distribution after data augmentation

Fig. 3. The distribution of CVEs in ESV class before and after data augmentation.



(a) CVE distribution before data augmenataion

(b) CVE distribution after data augmentation

Fig. 4. The distribution of CVEs in GPSV class before and after data augmentation.

Table 3
Balanced training data set.

	Original data	Augmented data
Train	26,000	38,000

The 200-sample threshold was empirically determined from the post-augmentation quartile distributions observed in Figs. 3(b) and 4(b), where the majority of keywords fell within the 50–1000 description range. This approach effectively balanced granular preservation with computational efficiency.

Table 3 demonstrates the data after applying these techniques, achieving balance while maintaining critical linguistic diversity for robust vulnerability classification.

4. Evaluation

4.1. Dataset

The vulnerability data in NVD (NVD, 2024) from 1999 to 2024, CNNVD (CNNVD, 2024) from 1999 to 2024, VarIoT (VarIoT, 2024) from 2008 to 2024, and CNVD (CNVD, 2024) from 2015 to 2020 are used to conduct the experimental research on automatic vulnerability classification. A script is used to categorize ESV and GPSV as detailed in Section 3.4.1, while ambiguous vulnerabilities (AV) are manually labeled. The description of 47,991 vulnerability reports is extracted, 170 of them are manually collected from other databases such as ICS-CERT (ICS-CERT, 2024), ZDI (ZDI, 2024), and Fortiguard (FortiGuard, 2024) specifically to be challenging for the models. A vulnerability description is a concise summary of the vulnerability. Table 4 shows an example of vulnerability descriptions from the NVD database and their corresponding categories. NVD and VarIoT vulnerability descriptions are in English, while CNVD and CNNVD are in Chinese. Therefore, we translated them into English using Google Translate Python API.

The 43,821 vulnerabilities extracted from NVD, CNNVD, CNVD, and VarIoT are divided into train, validation, and test sets. The base training dataset is extracted from NVD while the CVEs duplicates in other databases with different description styles are used as augmentation data resulting in 38,000 samples in the training dataset which are further divided into 80% train and 20% validation. While the vulnerabilities unique to each database are reserved to test the models. Table 5 depicts the 3 datasets and their number of vulnerability descriptions.

4.2. Parameter configuration

In this paper, we trained a domain-specific word2vec to better catch domain-specific word similarity. Gensim tools are used to train the word2vec model in 659,706 vulnerability descriptions. The architecture used is continuous skip-gram and the parameters are shown in Table 6.

The hyperparameters used for BiLSTM and BiGRU are shown in Table 7

4.3. Evaluation metrics

Vulnerability classification is treated as a multi-class classification task. The definitions for each component are as follows:

- TP_i (True Positive): Number of vulnerabilities correctly classified as category i.
- FP_i (False Positive): Number of vulnerabilities not in category i but incorrectly classified as i.
- FN_i (False Negative): Number of vulnerabilities that belong to category i but are classified under other categories.
- TN_i (True Negative): Number of vulnerabilities not in category i and correctly classified as not i.

Table 4
Vulnerability descriptions and their corresponding category.

CVE-ID	Description	Category
CVE-2018–14713	Format string vulnerability in appGet.cgi on ASUS RT-AC3200 version 3.0.0.4.382.50010 allows attackers to read arbitrary sections of memory and CPU registers via the "hook" URL parameter.	ESV
CVE-2021–44833	The CLI 1.0.0 for Amazon AWS OpenSearch has weak permissions for the configuration file.	GPSV
CVE-2023–0755	The affected products are vulnerable to improper validation of array index, which could allow an attacker to crash the interrupt server and remotely execute arbitrary code.	AV

ESV: Embedded System Vulnerability.
GPSV: General-purpose System Vulnerability.
AV: Ambiguous Vulnerability.

Table 5
Overview of the dataset.

Database	ESV	GPSV	AV	Total
Train set	10,169	10,138	10,093	30,400
Validation set	2,552	2,528	2,520	7,600
Unseen	59	57	54	170
CNNVD	866	272	520	1,658
CNVD	1,172	980	520	2,672
VarIoT	948	24	520	1,492

Table 6
Word2Vec model parameter configuration.

Hyperparameter	Values
Vector size	300
Context window size	5
Minimum word frequency	1
Workers	16

Table 7
Model hyperparameters.

Hyperparameter	BiLSTM/BiGRU
Batch-size	16, 32, 64, 128
Epochs	100
Layers	8
Dropout	0.3
Loss function	`sparse_categorical_crossentropy`
Optimizer	Adam
Learning rate	0.0001
Early Stop/patience	`val_loss/5`
Kernel initializer	`he_uniform`
Kernel regularizer	`l2(0.01)`

To evaluate the overall classification performance across all vulnerability categories, the following metrics are used: Accuracy (ACC), precision (P), recall (R), and F1-score ($F1$). The formulas for these metrics are:

$$\mathrm{ACC} = \frac{\mathrm{TP} + \mathrm{TN}}{(\mathrm{TP} + \mathrm{FP} + \mathrm{FN} + \mathrm{TN})} \tag{7}$$

$$P = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}} \tag{8}$$

$$R = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}} \tag{9}$$

$$F1 = \frac{2 \cdot P \cdot R}{P + R} \tag{10}$$

In addition, we used the AUC-ROC (Area Under the Receiver Operating Characteristic Curve) to assess the model's classification performance. The AUC-ROC is valued for its ability to separate predictive accuracy from threshold-specific operating conditions, offering a general measure of how well the model distinguishes between classes. The AUC score ranges from 0 to 1, where a score of 1.00 indicates excellent classification, while a score below 0.60 is considered poor, signifying performance close to random guessing.

We also incorporated the Matthews Correlation Coefficient (MCC) as an evaluation metric. The MCC accounts for all four quadrants of the confusion matrix and provides a balanced measure, even for datasets with imbalanced classes. The MCC is defined by the following equation:

$$\mathrm{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \tag{11}$$

The MCC value ranges from −1 to +1, with +1 indicating perfect prediction, 0 representing no better than random prediction, and −1 reflecting total disagreement between the predictions and the actual outcomes.

4.4. Experimental setup

A series of experiments were carried out to determine the effectiveness of our proposed word weighting approach for vulnerability classification. These experiments were executed on a computer equipped with double Xion CPU running at 2.40 GHz each and 62 GB of RAM, complemented by NVIDIA Geforce RTX 3090 GPUs with 24 GB of VRAM. The operating system used was Ubuntu 22.04 LTS. Throughout our experiments, we made use of Tensorflow version 2.12.1 in conjunction with Keras version 2.12.0.

4.5. Experiments & analysis

4.5.1. Hyperparameters

One of the important hyperparameters when creating a text classification model is sentence length. BiLSTM and BiGRU expect a fixed sentence length, therefore we compared 3 of the most common sentence lengths, 128, 256, and 512 and their effect on model performance, the result of the comparison using BiLSTM model is shown in Fig. 5. The model performance reached its highest with a sentence length of 256 with and accuracy of 97.63% and a training time of 64.15 min, 512 sentence length has the fastest training time but with an accuracy of 94.67, while 128 takes the longest training time with the lowest accuracy of 85.20%. Therefore, considering the accuracy and training time, the max length is set to 256.

4.5.2. Ablation analysis

The ablation analysis, illustrated in Fig. 6, provides comprehensive insights into how different architectures perform under varying batch sizes (16, 32, 64, and 128). Six architectures were evaluated:

A1: Word2Vec + TF–IDF + BiLSTM
A2: Word2Vec + TF–IDF + BiGRU
A3: Word2Vec + BiGRU

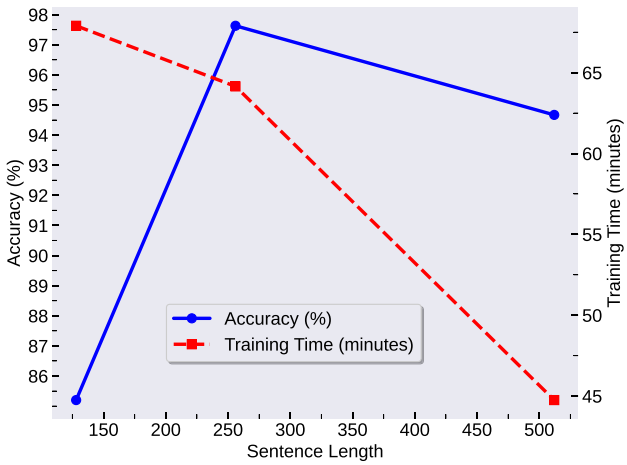


Fig. 5. Training accuracy and training time according to sentence lengths.

A4: Word2Vec + BiLSTM
A5: Word2Vec + Bayesian weighting + BiGRU
A6: Word2Vec + Bayesian weighting + BiLSTM

For the baseline configurations (A1 and A2) that rely on TF–IDF for word weighting, the performance remains moderate. Although A2 (with BiGRU) slightly outperforms A1 (with BiLSTM) across all batch sizes, neither variant reaches top-tier performance levels. Their accuracy, precision, recall, and F1 scores hover in the low-to-mid 60%, with relatively modest gains as batch sizes change.

In contrast, the models that use unweighted Word2Vec embeddings (A3 and A4) exhibit a clear upward trend with larger batch sizes. Both A3 and A4 improve incrementally, with A4 (BiLSTM) generally achieving higher accuracy and F1-scores reaching up to approximately 72% accuracy at a batch size of 128 suggesting that the BiLSTM configuration is better at capturing sequential dependencies when raw embeddings are employed.

The most significant improvements are observed with the incorporation of the proposed Bayesian word weighting mechanism. Both A5 and A6 consistently outperform their TF–IDF and unweighted counterparts. Notably, A6 (Word2Vec + Bayesian weighting + BiLSTM) reaches an impressive accuracy of 97.63%, precision of 97.84%, recall of 97.53%, and an F1-score of 97.60% at a batch size of 16, accompanied by an AUC-ROC of 98.78% and an MCC of 0.9652. Although A6's performance experiences a slight decline at larger batch sizes (with accuracies around 94%–95%), its metrics remain superior. Similarly, A5 (Word2Vec + Bayesian weighting + BiGRU) shows robust performance, its accuracy peaks at 93.49% with a batch size of 128 but it consistently trails A6.

Overall, these results demonstrate that the novel Bayesian word weighting algorithm, especially when coupled with domain-specific Word2Vec embeddings and a BiLSTM classifier, provides substantial performance gains in vulnerability classification. While batch size tuning affects all architectures, models employing Bayesian weighting maintain a distinct advantage by achieving higher accuracy, F1-score, and more robust AUC-ROC and MCC values across different batch sizes. This underscores the method's scalability and efficacy, even when the training dynamics are altered by varying batch sizes.

4.6. Comparative analysis

To evaluate the effectiveness of our proposed word weighting method, we conducted a comparative analysis with the TF-IGM by Chen et al. (2016) and N-TF–IDF weighting algorithm recently introduced by Wang et al. (2023), as well as different transformer-based models such as BERT and ModernBERT. TF-IGM and N-TF–IDF algorithms has demonstrated superior performance over traditional TF–IDF in vulnerability type classification. However, as our classification task and dataset differ from those in Wang et al. and Chen et al. study, a direct comparison of the methods is not feasible. To address this, we implemented Wang et al. and Chen et al. algorithms and applied them to our classification task and dataset. To simulate real world evaluation of the models, we evaluated them in four datasets from different databases.

Our evaluation demonstrates the proposed Bayesian weighting superiority over transformer-based models (BERT, DistilBERT, RoBERTa, XLNet, ModernBERT) and traditional weighting approaches (TF-IGM, N-TF–IDF) across four cybersecurity datasets. As shown in Table 8, our approach achieves state-of-the-art performance, particularly in generalization to unseen data where it attains 97.63% accuracy (+23.67% over BERT) and 0.9652 MCC. This performance dominance stems from four key factors: (1) adaptive term weighting aligned with cybersecurity semantics, (2) handling intra-class word importance effectively, (3) word-category association, and (4) effective generalization beyond training distributions.

Transformer-based models underperform due to their reliance on contextual attention mechanisms and substantial data requirements. While effective for general language tasks, these models struggle with subtle distinctions between vulnerability classes in domain-specific texts. Their performance degrades significantly on the unseen dataset (59.76% accuracy for ModernBERT which achieved promising results in the other datasets vs. 97.63% for our method), highlighting limitations in cross-domain generalization. The VarIoT dataset's class imbalance further exacerbates these issues, with ModernBERT showing 0.8055 MCC compared to 0.9113 achieved by our approach.

Traditional weighting methods, such as TF-IGM and N-TF–IDF, face significant limitations due to the static term weighting across different classes. While TF-IGM overall achieved better results than N-TF–IDF due to its reliance on inverse gravity moment, which assigns higher weights to rare but discriminative words, its fixed weights for the same word across different classes restrict its effectiveness. This results in a 10.63% lower accuracy on CNVD compared to our proposed approach, which achieved the highest performance. On the other hand, N-TF–IDF's emphasis on class-frequency words makes it more vulnerable to data bias during training, leading to reduced performance. Specifically, it performs 19.2% worse than our method in terms of F1-score on CNVD, highlighting the need for dynamic, class-conditioned weighting.

The superior performance of our method is largely attributable to its Bayes-based weighting, which enhances the influence of discriminative technical terms by dynamically adjusting word importance based on their class-specific power. This is further bolstered by a custom-trained word2vec model that effectively captures semantic similarities in cybersecurity terminology, ensuring that even unseen words during training are properly represented. In contrast, baseline methods like TF-IGM and N-TF–IDF use a uniform weighting scheme that does not account for class-specific variations, while transformer-based models rely heavily on contextual information, a reliance that becomes a limitation when facing overlapping class contexts.

4.6.1. Resource-performance trade-offs

Fig. 7 reveals our method's higher resource demands (12.38 GB GPU memory vs. 2.99 GB for ModernBERT) and longer training times (65.79 vs. 9.34 min). However, these costs are justified by critical performance gains. While DistilBERT and TF-IGM offer lower resource usage, their accuracy deficits make them unsuitable for production-grade classification systems. These results validate the effectiveness of integrating Bayesian weighting with domain-specific embeddings for cybersecurity text classification. By dynamically adjusting the importance of technical terms according to class-specific distributions, our method not only enhances classification performance but also overcomes the limitations observed in traditional weighting methods and transformer-based models, particularly in handling nuanced semantics and unseen data. This approach lays a promising foundation for future work in related areas such as threat intelligence extraction and attack pattern recognition.

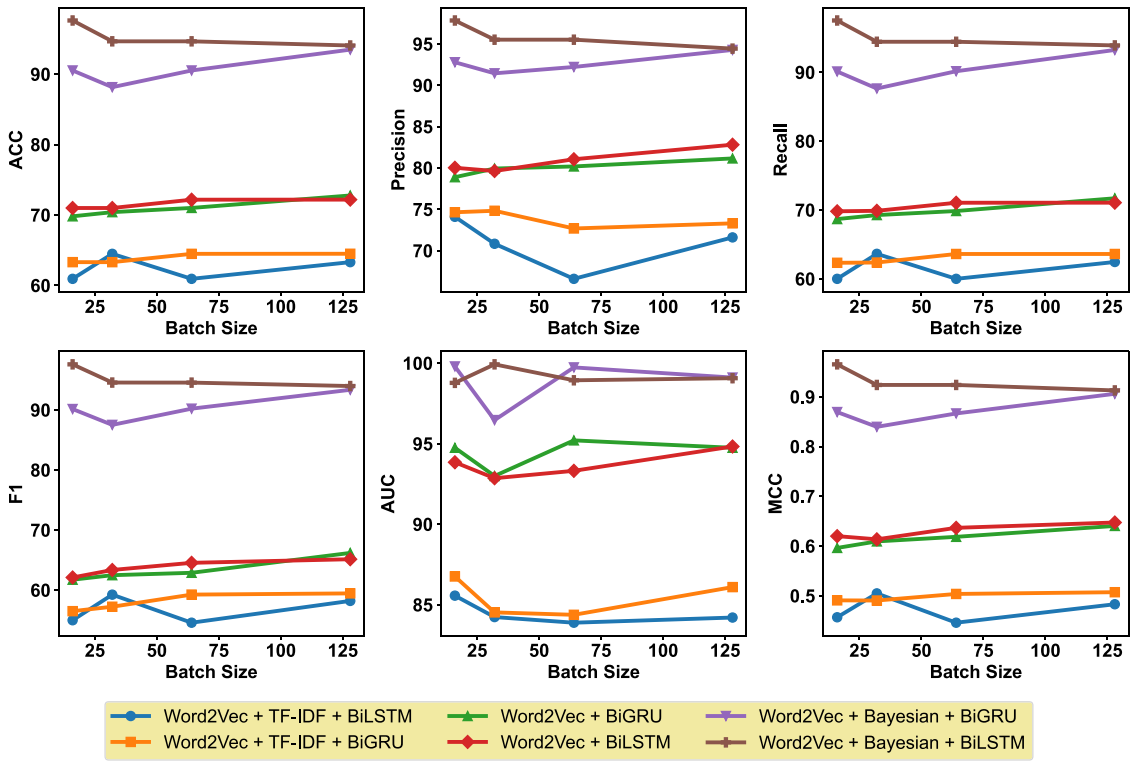


Fig. 6. Model performance across varying batch sizes.

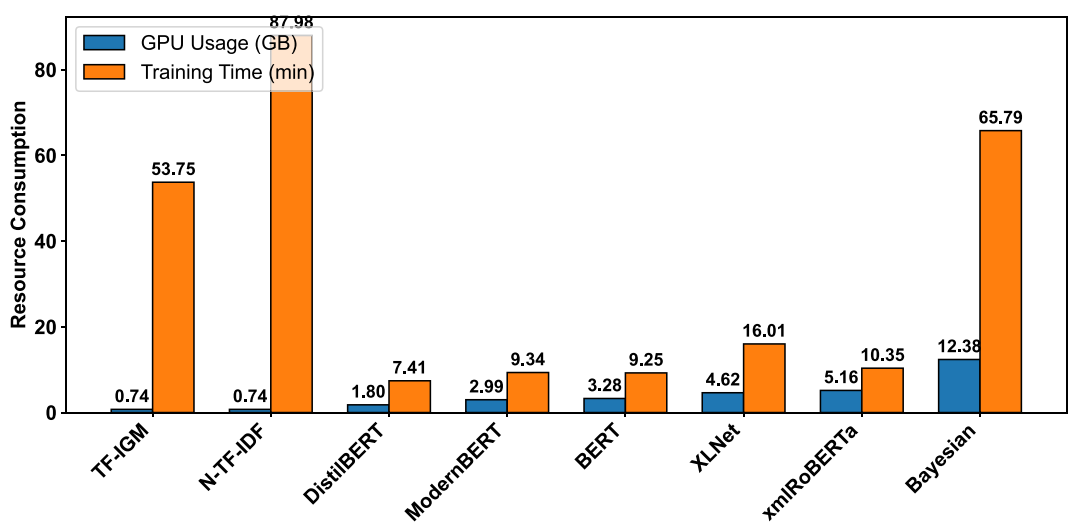


Fig. 7. Model resource consumption proportional to training time.

5. Limitations and future work

While the proposed method advances vulnerability classification, several key limitations warrant consideration:

- **Out-of-Vocabulary (OOV) Handling:** The reliance on pre-trained domain-specific embeddings limits robustness to novel technical terms of emerging technologies not present in the training corpus.
- **Static Frequency Assumptions:** The Bayesian weighting mechanism relies on words posterior probabilities, which might hinder its effectiveness if the training data is highly biased or not sufficiently representative.
- **Computational Overhead:** The BiLSTM architecture incurs higher inference latency compared to transformer-based models, which hinders real-time deployment in large-scale vulnerability databases.

To address these limitations, we propose the following research avenues:

- **Hybrid Embedding Architectures:** Integrate transformer-based embeddings with domain-specific embeddings to mitigate OOV issues by leveraging their ability to handle OOV words through special tokenizing.
- **Context-Aware Weighting:** Augment Bayesian probabilistic weighting metrics with attention mechanisms to dynamically adjust term importance based on syntactic and semantic context.

Table 8
Performance comparison of the models on cybersecurity datasets.

Model	Metrics (%, except MCC)					
	Acc.	Prec.	Rec.	F1	AUC	MCC
VarIoT dataset						
BERT	79.83	65.73	74.78	57.76	94.27	0.6417
DistilBERT	77.55	64.69	75.25	55.48	96.92	0.5954
ROBERTA	85.52	70.46	84.03	69.94	97.34	0.7072
XLNET	88.54	85.08	86.74	84.49	**99.07**	0.7638
MODERNBERT	90.62	83.80	80.72	81.64	98.75	0.8055
N-TF–IDF	69.43	54.75	65.62	46.40	87.72	0.3900
TF-IGM	78.41	63.83	71.98	56.89	93.77	0.5764
Proposed work	**95.77**	**97.55**	**96.22**	**96.80**	96.60	**0.9113**
CNVD dataset						
BERT	88.40	90.36	81.41	82.87	97.19	0.8235
DistilBERT	87.31	89.46	79.34	80.41	99.18	0.8074
ROBERTA	91.77	93.53	86.73	88.68	99.73	0.8732
XLNET	93.30	95.10	89.25	91.23	99.70	0.8973
MODERNBERT	94.84	96.24	91.69	93.40	**99.85**	0.9204
N-TF–IDF	82.52	85.62	72.25	71.61	95.69	0.7326
TF-IGM	87.01	88.93	79.40	80.81	98.41	0.8001
Proposed work	**97.64**	**97.88**	**96.28**	**97.01**	98.51	**0.9630**
CNNVD dataset						
BERT	82.21	85.24	79.75	79.26	96.05	0.7215
DistilBERT	79.73	81.84	76.91	74.93	97.61	0.6843
ROBERTA	86.67	84.50	84.46	82.20	98.86	0.7932
XLNET	89.51	86.43	87.94	85.27	98.87	0.8395
MODERNBERT	91.92	88.84	90.88	88.55	**99.50**	0.8749
N-TF–IDF	73.76	75.55	70.74	63.65	94.00	0.6089
TF-IGM	80.39	79.82	78.06	74.24	97.58	0.7002
Proposed work	**96.20**	**93.69**	**95.43**	**94.32**	98.79	**0.9384**
Unseen dataset						
BERT	73.96	83.12	72.97	68.66	86.67	0.6632
DistilBERT	67.46	80.74	66.51	62.03	91.74	0.5780
ROBERTA	63.91	76.32	62.87	55.91	84.15	0.5126
XLNET	61.54	73.70	60.51	52.95	83.55	0.4667
MODERNBERT	59.76	73.14	58.66	49.23	88.35	0.4446
N-TF–IDF	69.23	78.82	68.10	61.41	91.77	0.5878
TF-IGM	76.33	83.91	75.35	71.41	96.73	0.6920
Proposed work	**97.63**	**97.84**	**97.53**	**97.60**	**98.78**	**0.9652**

Note: Metrics (Acc., Prec., Rec., F1, AUC) are reported in percentages. MCC is reported as a decimal.

- **Lightweight Deployment:** Explore knowledge distillation techniques to compress the BiLSTM architecture while retaining 98%+ accuracy, targeting a 5 times reduction in inference latency.

These improvements aim to transform the framework into a deployable system for real-time vulnerability prioritization in security operations centers (SOCs), while broadening its applicability to adjacent cybersecurity tasks such as exploitability prediction and CVE severity scoring.

6. Conclusion

To enhance the security of critical infrastructure embedded devices and reduce the cost of vulnerability management, this paper proposes a novel word-weighting technique based on Bayes' theorem and domain-specific word embeddings for vulnerability classification. A dataset was constructed using vulnerability descriptions extracted from various databases, including NVD, CNNVD, and ICS-CERT. By combining the Bayes-based word-weighting method with the Word2Vec model, trained on custom cybersecurity data, a weighted word embedding vector is generated, and a BiLSTM/BiGRU classifier is trained to construct a domain-specific vulnerability classification framework.

The effectiveness of the proposed approach was validated through rigorous testing, including ablation studies and comparative analyses. Our proposed weighting method, combined with the domain-specific Word2Vec model, outperformed other methods, achieving an accuracy of 97.63%, and an F1-score of 97.60% on the manually collected test data.

CRediT authorship contribution statement

Aissa Ben Yahya: Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Hicham El Akhal:** Writing – review & editing, Conceptualization. **El Mehdi Ismaili Alaoui:** Writing – review & editing. **Abdelbaki El Belrhiti El Alaoui:** Writing – review & editing, Writing – original draft, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This research received no external funding.

Data availability

National Vulnerability Database [online]. Available: 1as https://nvd.nist.gov/vuln/data-feeds

Chinese National Vulnerability Database [online]. Available: https://www.cnnvd.org.cn/home/dataDownLoad

China National Vulnerability Database [online]. Available: https://www.cnvd.org.cn/shareData/download/243 from [243 to 1531]

VarIoT [online]. Available: https://www.variotdbs.pl/vulns/

Train, Validation, Test datasets and the code implementation of the Bayesian weighting algorithm used in this study is available [online]. Available: https://github.com/aissa302/Bayesian-weighting-framework.git.

References

Aghaei, E., Shadid, W., Al-Shaer, E., 2020. Threatzoom: CVE2CWE using hierarchical neural network. arXiv preprint arXiv:2009.11501.

Ben Yahya, A., El Akhal, H., El Alaoui, A.E.B., 2024. Machine learning-based collection and analysis of embedded systems vulnerabilities. In: Enhancing Performance, Efficiency, and Security Through Complex Systems Control. IGI Global, pp. 242–261.

Blinowski, G.J., Piotrowski, P., 2020. CVE based classification of vulnerable IoT systems. In: International Conference on Dependability and Complex Systems. Springer, pp. 82–93, URL: https://doi.org/10.1007/978-3-030-48256-5_9.

Bojanowski, P., Grave, E., Joulin, A., Mikolov, T., 2017. Enriching word vectors with subword information. Trans. Assoc. Comput. Linguist. 5, 135–146.

Chen, J., Kudjo, P.K., Mensah, S., Brown, S.A., Akorfu, G., 2020. An automatic software vulnerability classification framework using term frequency-inverse gravity moment and feature selection. J. Syst. Softw. 167, 110616, URL: https://doi.org/10.1016/j.jss.2020.110616.

Chen, K., Zhang, Z., Long, J., Zhang, H., 2016. Turning from TF-IDF to TF-IGM for term weighting in text classification. Expert Syst. Appl. 66, 245–260.

CNNVD, 2024. Chinese national vulnerability database. URL: https://www.cnnvd.org.cn/. (Accessed November 2024).

CNVD, 2024. China national vulnerability database. URL: https://www.cnvd.org.cn/. (Accessed November 2024).

Davari, M., Zulkernine, M., Jaafar, F., 2017. An automatic software vulnerability classification framework. In: 2017 International Conference on Software Security and Assurance. ICSSA, IEEE, pp. 44–49.

EMB3D, 2024. Mitre emb3d. URL: https://emb3d.mitre.org/. (Accessed November 2024).

Ezenwoye, O., Pinconschi, E., 2024. A study of fine-tuned language models in vulnerability classification. In: 2024 12th International Symposium on Digital Forensics and Security. ISDFS, IEEE, pp. 1–6.

FortiGuard, 2024. Fortiguard. URL: https://www.fortiguard.com/. (Accessed November 2024).

Gillioz, A., Casas, J., Mugellini, E., Abou Khaled, O., 2020. Overview of the transformer-based models for NLP tasks. In: 2020 15th Conference on Computer Science and Information Systems. FedCSIS, IEEE, pp. 179–183.

Google translate API, 2024. Google translate API. URL: https://pypi.org/project/googletrans/. (Accessed November 2024).

Guo, B., Zhang, C., Liu, J., Ma, X., 2019. Improving text classification with weighted word embeddings via a multi-channel TextCNN model. Neurocomputing 363, 366–374.

Han, Z., Li, X., Xing, Z., Liu, H., Feng, Z., 2017. Learning to predict severity of software vulnerability using only vulnerability description. In: 2017 IEEE International Conference on Software Maintenance and Evolution. ICSME, IEEE, pp. 125–136.

Huang, G., Li, Y., Wang, Q., Ren, J., Cheng, Y., Zhao, X., 2019. Automatic classification method for software vulnerability based on deep neural network. IEEE Access 7, 28291–28298, URL: https://doi.org/10.1109/ACCESS.2019.2900462.

ICS-CERT, 2024. Industrial control systems cyber emergency response team. URL: https://www.cisa.gov/. (Accessed November 2024).

Jian, L., Purui, S., Min, Y., Liang, H., Yuan, Z., Xueyang, Z., Huimin, L., 2017. Software and cyber security-a survey. J. Softw. 29 (1), 42–68.

Kota, K., Manjunatha, A., et al., 2024. CWE prediction using CVE description-the semantic similarity approach. Procedia Comput. Sci. 235, 1167–1178.

Kudjo, P.K., Chen, J., Zhou, M., Mensah, S., Huang, R., 2019. Improving the accuracy of vulnerability report classification using term frequency-inverse gravity moment. In: 2019 IEEE 19th International Conference on Software Quality, Reliability and Security. QRS, IEEE, pp. 248–259, URL: https://10.1109/QRS.2019.00041.

Liu, Z., 2025. Intelligent classification of computer vulnerabilities and network security management system: Combining memristor neural network and improved TCNN model. PloS One 20 (1), e0318075.

Marali, M., Dhanalakshmi, R., Rajagopalan, N., 2024. A hybrid transformer-based BERT and LSTM approach for vulnerability classification problems. Int. J. Math. Oper. Res. 28 (3), 275–295.

Mikolov, T., Chen, K., Corrado, G., Dean, J., 2013. Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781.

Nakagawa, S., Nagai, T., Kanehara, H., Furumoto, K., Takita, M., Shiraishi, Y., Takahashi, T., Mohri, M., Takano, Y., Morii, M., 2019. Character-level convolutional neural network for predicting severity of software vulnerability from vulnerability description. IEICE Trans. Inf. Syst. 102 (9), 1679–1682.

NVD, 2024. National vulnerability database. URL: https://nvd.nist.gov/. (Accessed November 2024).

Papp, D., Ma, Z., Buttyan, L., 2015. Embedded systems security: Threats, vulnerabilities, and attack taxonomy. In: 2015 13th Annual Conference on Privacy, Security and Trust. PST, ieee, pp. 145–152, URL: https://doi.org/10.1109/PST.2015.7232966.

Pennington, J., Socher, R., Manning, C.D., 2014. Glove: Global vectors for word representation. In: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing. EMNLP, pp. 1532–1543.

Salton, G., Buckley, C., 1988. Term-weighting approaches in automatic text retrieval. Inf. Process. Manage. 24 (5), 513–523.

Shahid, M.R., Debar, H., 2021. Cvss-bert: Explainable natural language processing to determine the severity of a computer security vulnerability from its description. In: 2021 20th IEEE International Conference on Machine Learning and Applications. ICMLA, IEEE, pp. 1600–1607.

Sharma, R., Sibal, R., Sabharwal, S., 2021. Software vulnerability prioritization using vulnerability description. Int. J. Syst. Assur. Eng. Manag. 12, 58–64.

Spanos, G., Angelis, L., 2018. A multi-target approach to estimate software vulnerability characteristics and severity scores. J. Syst. Softw. 146, 152–166.

Sun, X., Li, L., Bo, L., Wu, X., Wei, Y., Li, B., 2024. Automatic software vulnerability classification by extracting vulnerability triggers. J. Softw.: Evol. Process. 36 (2), e2508.

Tian, H., Wu, L., 2018. Microblog emotional analysis based on TF-IWF weighted Word2vec model. In: 2018 IEEE 9th International Conference on Software Engineering and Service Science. ICSESS, IEEE, pp. 893–896.

VarIoT, 2024. VarIoT. URL: https://www.variotdbs.pl/vulns/. (Accessed November 2024).

Vaswani, A., 2017. Attention is all you need. Adv. Neural Inf. Process. Syst..

Vishnu, P., Vinod, P., Yerima, S.Y., 2022. A deep learning approach for classifying vulnerability descriptions using self attention based neural network. J. Netw. Syst. Manage. 30, 1–27.

Wang, Q., Gao, Y., Ren, J., Zhang, B., 2023. An automatic classification algorithm for software vulnerability based on weighted word vector and fusion neural network. Comput. Secur. 126, 103070.

Wang, J.A., Guo, M., 2010. Vulnerability categorization using Bayesian networks. In: Proceedings of the Sixth Annual Workshop on Cyber Security and Information Intelligence Research. pp. 1–4, URL: https://doi.org/10.1145/1852666.1852699.

Wang, Q., Li, Y., Wang, Y., Ren, J., 2022. An automatic algorithm for software vulnerability classification based on CNN and GRU. Multimedia Tools Appl. 81 (5), 7103–7124.

Wen, T., Zhang, Y., Wu, Q., Yang, G., 2015. ASVC: An automatic security vulnerability categorization framework based on novel features of vulnerability data. J. Commun. 10 (2), 107–116.

ZDI, 2024. Zero-day initiative. URL: https://www.zerodayinitiative.com/. (Accessed November 2024).

Zhu, C., Du, G., Wu, T., Cui, N., Chen, L., Shi, G., 2022. Bert-based vulnerability type identification with effective program representation. In: International Conference on Wireless Algorithms, Systems, and Applications. Springer, pp. 271–282.